Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas

ANALISIS DE SISTEMAS II

Octavo Semestre

PROYECTO FINAL

Docente: Ing. Edson Daniel Ruano Carias

Julio César Sandoval Benitez Carné: 0907-23-19529

Ariatna Odeth Berganza Villatoro Carné: 0907-23-5837

Katherine Daniela Portillo Melendez Carné: 0907-23-8051

Juan Carlos Manuel Lechuga Ortiz Carné: 0907-23-3626

Jorge Danilo Ucelo Román Carné: 0907-23-18008

Guatemala, 2026

Repositorio: https://github.com/DeuceloR18/ProyectoFinalAnalisisII.git

Integrante	Responsabilidad principal
Katherine Daniela Portillo Melendez	Requerimientos y SDD: análisis, especificaciones, planes, tareas y trazabilidad
Ariatna Odeth Berganza Villatoro	Backend: capas Domain y Application
Jorge Danilo Ucelo Román	Infrastructure, base de datos
Julio César Sandoval Benitez	base de datos y API
Juan Carlos Manuel Lechuga Ortiz	Interfaz web, UX y diagramas C4

Resumen

El proyecto atiende el problema de administrar turnos de atención en oficinas, clínicas pequeñas y centros de servicios. Una aplicación web permite al recepcionista registrar servicios, generar turnos con número correlativo, consultar la fila de espera, llamar al siguiente turno, finalizar la atención y cancelar turnos en espera. El desarrollo siguió la cadena problema, requerimiento, especificación, diseño, tarea, implementación y prueba, apoyada en GitHub Spec Kit y GitHub Copilot.

A. Análisis

A.1 Problema y contexto

[COMPLETAR: copiar el párrafo del problema de la propuesta aprobada.] Sin un control de turnos, la atención se desordena, se pierde el orden de llegada y no queda registro de quién fue atendido. El sistema propone un flujo mínimo viable que resuelve ese problema para un recepcionista que atiende varios servicios.

A.2 Actores

Actor	Tipo	Relación con el sistema
Recepcionista	Principal	Usa todas las funciones: registra servicios, genera turnos, consulta la fila, llama, finaliza y cancela
Cliente	Secundario	No usa el sistema; recibe su número de turno y espera ser llamado

A.3 Requerimientos

Requerimiento	Descripción	Prioridad	Especificación
RF01	Registrar y consultar servicios (nombre y prefijo)	Alta	SPEC-01
RF02	Generar turno con número correlativo por servicio y por día	Alta	SPEC-02
RF03	Consultar la fila de espera de un servicio	Alta	SPEC-03
RF04	Llamar al siguiente turno	Alta	SPEC-04
RF05	Finalizar la atención de un turno	Alta	SPEC-05
RF06	Cancelar un turno en espera con confirmación	Secundaria	SPEC-06

RF01 a RF05 forman el flujo mínimo viable; RF06 es de prioridad secundaria.

A.4 Reglas de negocio

- El prefijo del servicio se guarda siempre en mayúscula, es único sin importar la capitalización y tiene un máximo de 5 caracteres. El nombre del servicio y el del cliente tienen un máximo de 100.
- El número de turno se forma con el prefijo, un guion y un correlativo de tres dígitos (por ejemplo C-001). El correlativo es independiente por servicio y se reinicia cada día.
- Estados de un turno: Esperando, En atención, Finalizado y Cancelado. Todo turno nuevo queda Esperando.
- Solo se llama al turno Esperando más antiguo del servicio. Un servicio solo puede tener un turno En atención a la vez.
- Solo un turno En atención puede finalizarse; solo un turno Esperando puede cancelarse, previa confirmación. Un turno Finalizado o Cancelado no cambia de estado.
- Un número cancelado no se reutiliza.
- La información se conserva al cerrar y volver a abrir la aplicación.

A.5 Casos de uso

Caso de uso	Actor	Requerimiento
CU-01 Registrar servicio	Recepcionista	RF01
CU-02 Consultar servicios	Recepcionista	RF01

Caso de uso	Actor	Requerimiento
CU-03 Generar turno	Recepcionista	RF02
CU-04 Consultar fila de espera	Recepcionista	RF03
CU-05 Llamar siguiente turno	Recepcionista	RF04
CU-06 Finalizar atención	Recepcionista	RF05
CU-07 Cancelar turno	Recepcionista	RF06

Ejemplo, CU-05 Llamar siguiente turno. Precondición: el servicio existe. Flujo principal: el recepcionista selecciona el servicio y pulsa «Llamar siguiente»; el sistema toma el turno Esperando más antiguo, lo pasa a En atención y lo muestra como turno actual. Flujos alternos: si no hay turnos en espera, el sistema informa «No hay turnos en espera para este servicio» sin cambiar nada; si ya hay un turno En atención, informa que debe finalizarse antes de llamar a otro y no modifica ningún turno.

[COMPLETAR: incluir los demás casos de uso con flujo principal y alternos desde docs/analisis/casos-de-uso.md, y las historias de usuario desde historias-de-usuario.md.]

B. Especificaciones SDD

B.1 Enfoque y herramienta

Se utilizó GitHub Spec Kit (specify-cli 1.0.1) con GitHub Copilot en VS Code. Las especificaciones definen el comportamiento esperado, las reglas, las restricciones y los criterios de aceptación en formato Dado, Cuando y Entonces, y guiaron el plan técnico, las tareas, la implementación y las pruebas.

B.2 Constitution del proyecto

La constitution (.specify/memory/constitution.md) fija las reglas del equipo: ninguna funcionalidad sin especificación aprobada, trazabilidad con identificadores SPEC, criterios de aceptación verificables, arquitectura en capas con Domain independiente, alcance limitado al flujo mínimo viable, una prueba por criterio, UX clara y accesible, uso responsable de IA, documentación en español y flujo de Git con ramas cortas.

B.3 Especificaciones utilizadas

Especificación	Requerimiento	Criterios de aceptación	Archivos
SPEC-01 Registrar y	RF01	8	spec.md, plan.md,

Especificación	Requerimiento	Criterios de aceptación	Archivos
consultar servicios			tasks.md
SPEC-02 Generar turno	RF02	12	spec.md, plan.md, tasks.md
SPEC-03 Consultar fila de espera	RF03	10	spec.md, plan.md, tasks.md
SPEC-04 Llamar siguiente turno	RF04	10	spec.md, plan.md, tasks.md
SPEC-05 Finalizar atención	RF05	9	spec.md, plan.md, tasks.md
SPEC-06 Cancelar turno	RF06	9	spec.md, plan.md, tasks.md

Total: 58 criterios de aceptación. Cada especificación está en su carpeta dentro de specs/.

B.4 Evidencia del proceso SDD

1. La constitution se generó con la skill speckit-constitution y se ajustó por el equipo.
2. Cada especificación se generó con speckit-specify a partir de una descripción redactada por el equipo con base en la propuesta aprobada.
3. Cada plan se generó con speckit-plan, y cada lista de tareas con speckit-tasks, en ese orden.
4. Las especificaciones y luego el plan y las tareas se subieron por Pull Request y fueron revisados por el equipo antes del merge.

B.5 Decisiones del equipo durante el proceso

Situación	Decisión
El plan de SPEC-01 detectó que la especificación no fijaba una longitud máxima para el prefijo y propuso 450 caracteres	El equipo definió 5 caracteres para el prefijo y 100 para el nombre. Se actualizó primero la especificación, con su regla y criterio, y luego el plan
Mayúsculas en el prefijo	Se guarda siempre en mayúscula y la unicidad no distingue capitalización

Situación	Decisión
Turno En atención por servicio	Cada servicio tiene su propio turno actual
Número cancelado	No se reutiliza
Confirmación al cancelar	Es una regla de la interfaz; la API recibe la orden ya confirmada
Idioma	Las especificaciones y la documentación se dejaron solo en español

B.6 Trazabilidad

La tabla docs/trazabilidad/trazabilidad.md relaciona cada criterio de aceptación con su requerimiento, sus tareas, su prueba prevista, su resultado y su evidencia. La cadena completa por requerimiento está en la sección G.

C. Diseño

C.1 Componentes y responsabilidades

Componente	Responsabilidad	Depende de
SistemaTurnos.Domain	Entidades Servicio y Turno, estados y transiciones de estado	Nada
SistemaTurnos.Application	Casos de uso, servicios e interfaces de repositorio	Domain
SistemaTurnos.Infrastructure	Acceso a datos con SQL Server y Dapper; implementa los repositorios	Application y Domain
SistemaTurnos.Web	API REST e interfaz web estática	Application e Infrastructure
Base de datos SQL Server	Persistencia de servicios y turnos, con restricciones de unicidad	

C.2 Arquitectura y justificación

Se eligió una arquitectura en capas. Separa las reglas de negocio (Domain y Application) del acceso a datos y de la interfaz, lo que permite validar las transiciones de estado sin depender de la base de datos, cambiar la persistencia sin tocar las reglas y repartir el trabajo entre cuatro personas con pocas colisiones. Es proporcional al tamaño del problema: un sistema pequeño con reglas de estado claras no requiere una arquitectura más compleja. La independencia de Domain está exigida por la constitution.

C.3 Modelos C4

[COMPLETAR: insertar los diagramas de contexto, contenedores y componentes de docs/diseno y explicar en una o dos frases cada uno.]

C.4 API implementada

Ruta	Operación	Especificación
GET y POST api/servicios	Consultar y registrar servicios	SPEC-01
POST api/turnos	Generar turno	SPEC-02
GET api/turnos/espera/{servicioId}	Fila de espera	SPEC-03
POST api/turnos/siguiente/{servicioId} y GET api/turnos/actual/{servicioId}	Llamar siguiente y turno actual	SPEC-04
POST api/turnos/finalizar/{turnoId}	Finalizar atención	SPEC-05
POST api/turnos/cancelar/{turnoId}	Cancelar turno	SPEC-06

Las rutas implementadas pueden diferir de las propuestas en los planes; esa diferencia se declara en la sección E.

C.5 Experiencia de usuario

La interfaz es una sola pantalla con cuatro áreas: registrar servicio, generar turno, atención de turnos (turno actual, llamar siguiente y finalizar) y fila de espera con botón de cancelar. Aspectos considerados: flujo claro de arriba hacia abajo, mensajes de confirmación y de error, confirmación antes de cancelar con el número del turno, botones diferenciados por color y por acción, y consistencia visual. [COMPLETAR: incluir los wireframes de docs/ux y capturas de la pantalla.]

Aspectos de accesibilidad que quedan por mejorar: etiquetas visibles en los campos del formulario, y mostrar la posición de cada turno en la fila. Ver la sección E.5.

D. Aplicación

- Código fuente: [COMPLETAR: enlace del repositorio en GitHub].
- Tecnologías: C# y .NET 10, ASP.NET Core, SQL Server con Dapper, HTML, CSS y JavaScript.
- Flujo mínimo viable funcionando: registrar un servicio, generar turnos, consultar la fila, llamar siguiente, finalizar y cancelar.

Instrucciones para ejecutar

Las instrucciones completas están en el README.md del repositorio. En resumen:

1. Clonar el repositorio y compilar con dotnet build src\SistemaTurnos.sln.
2. Crear la base de datos ejecutando src\CrearBaseDatos.sql en SQL Server.
3. Definir la cadena de conexión en la variable de entorno ConnectionStrings__DefaultConnection, sin modificar appsettings.json.
4. Ejecutar dotnet run --project src\SistemaTurnos.Web y abrir en el navegador la dirección que indique la consola.

E. Verificación

E.1 Estrategia

Cada criterio de aceptación de las seis especificaciones tiene una prueba prevista en su plan.md. La verificación del sistema integrado se realizó con pruebas de aceptación manuales en pantalla, con capturas guardadas en docs/pruebas/capturas, y los resultados se registran en la tabla de trazabilidad.

E.2 Resultados

[ACTUALIZAR con los números finales de docs/trazabilidad/trazabilidad.md]

Estado	Cantidad de criterios
Aprobada, con captura de evidencia	[COMPLETAR]
Falla	[COMPLETAR]
No probado	[COMPLETAR]
Total	58

Cada criterio aprobado enlaza su captura en la columna Evidencia de la tabla de trazabilidad.

E.3 Pruebas del prototipo de interfaz

El repositorio incluye una interfaz de recepción alternativa (src/Turnos.Web) con pruebas automatizadas de navegador. Esas pruebas se ejecutan contra una API

simulada, por lo que no demuestran el comportamiento del backend real ni la persistencia. Esa interfaz no forma parte de la solución que se ejecuta.

E.4 Pruebas automatizadas del backend

Los planes prevén pruebas de Domain, de API y de integración con SQL Server, pero la solución no cuenta con un proyecto de pruebas automatizadas. Queda como trabajo pendiente.

E.5 Diferencias y pendientes

- Criterios no ejecutados o no verificables desde la interfaz (por ejemplo, reinicio diario del correlativo, correlativo C-1000 y operaciones simultáneas), registrados como No probado.
- Diferencias entre la interfaz y las especificaciones: [COMPLETAR, por ejemplo: la fila de espera no muestra la posición de cada turno; los campos no tienen etiqueta visible].
- Las rutas de la API implementada difieren de las propuestas en los planes. La interfaz entregada usa las rutas reales.

F. Uso de inteligencia artificial

F.1 Herramientas utilizadas

Herramienta	Para qué se usó
GitHub Spec Kit	Estructurar el proceso: constitution, especificación, plan y tareas
GitHub Copilot en VS Code (modo Agent)	Ejecutar las skills de Spec Kit y generar especificaciones, planes, tareas y la tabla de trazabilidad
Claude (chat)	Orientación sobre el proceso y redacción de las instrucciones enviadas a Copilot
[COMPLETAR: otras herramientas usadas por Personas 2, 3 y 4]	[COMPLETAR]

F.2 Cómo se trabajó

El equipo redactó la descripción del comportamiento esperado a partir de la propuesta aprobada, la pasó a la skill correspondiente, revisó lo generado contra la propuesta y las decisiones del equipo, corrigió lo que no coincidía y lo subió por Pull Request para revisión.

F.3 Decisiones y modificaciones del equipo

- Longitud del prefijo: la IA sugirió 450 caracteres por el límite del índice de SQL Server; el equipo definió 5 y 100 y actualizó primero la especificación.
- Idioma: las especificaciones y la constitution salieron con contenido en inglés; se dejaron solo en español.
- Alcance: Copilot propuso consultar documentación externa y explorar carpetas fuera del proyecto; el equipo lo rechazó para mantener las decisiones técnicas en manos del equipo.
- Los planes de SPEC-02 a SPEC-06 incorporaron decisiones del equipo: correlativo por servicio y por día, un turno En atención por servicio garantizado en la base de datos y número cancelado no reutilizable.
- [COMPLETAR: decisiones y modificaciones de Personas 2, 3 y 4 sobre el código generado o asistido por IA.]

F.4 Evidencia

La evidencia por integrante está en docs/evidencia-ia, con capturas de las interacciones más representativas.

F.5 Responsabilidad

La IA generó borradores. Las decisiones de alcance, las reglas de negocio, los límites y el código entregado son responsabilidad del equipo.

G. Coherencia entre problema, especificación, diseño, implementación y prueba

Requerimiento	Especificación	Diseño e implementación	Prueba y evidencia
RF01 Servicios	SPEC-01	Servicio en Domain; ServiciosController; tabla de servicios con prefijo único	Captura SPEC-01 y criterios en la trazabilidad
RF02 Generar turno	SPEC-02	Turno en Domain; POST api/turnos; correlativo por servicio y día	Captura SPEC-02
RF03 Fila de espera	SPEC-03	GET api/turnos/espera/{servicioId}	Captura SPEC-03
RF04 Llamar siguiente	SPEC-04	POST api/turnos/siguiente/{servicioId}; GET actual	Captura SPEC-04
RF05	SPEC-05	POST api/turnos/finalizar/{turnoId}	Captura

Requerimiento	Especificación	Diseño e implementación	Prueba y evidencia
Finalizar			SPEC-05
RF06 Cancelar	SPEC-06	POST api/turnos/cancelar/{turnoId}; confirmación en la interfaz	Captura SPEC-06

H. Conclusiones y trabajo pendiente

- El sistema cubre el flujo mínimo viable y las seis especificaciones a nivel funcional.
- El proceso SDD dejó especificaciones, planes, tareas y una tabla de trazabilidad que relacionan cada criterio con su verificación.
- Pendiente: pruebas automatizadas del backend, completar la evidencia de los criterios en estado No probado, mejorar la accesibilidad de la interfaz y mostrar la posición de cada turno en la fila.
- Fuera de alcance por no tener especificación aprobada: autenticación, reportes, pantalla pública para clientes y edición o eliminación de servicios.